ASCII characters. In contrast, POST sends form data within the body of the HTTP request, keeping it invisible in the URL and making it more secure for sensitive data like user registration, login, or file uploads. POST has no practical size limit and supports various data types, including binary data.

The `enctype` attribute determines how form data is encoded before submission, and is only relevant when `method="post"`. It supports values like `application/x-www-form-urlencoded` (default, for key-value data), `multipart/form-data` (required for file uploads), and `text/plain` (mainly for debugging). With `method="get"`, enctype is ignored because data is appended to the URL.

The `action` attribute specifies the destination URL where the form data is sent upon submission. This can be a relative path (like `/submit-form.php`) or an absolute URL (such as `https://example.com/process`). The `action` attribute determines which server-side script or endpoint will receive and process the form data. If omitted, the form submits to the current page by default. This approach can be useful if the same script or page is responsible for both displaying and processing the form, but it is generally recommended to explicitly specify the `action` attribute to ensure clarity and control over where form data is sent.

The `target` attribute defines where the response from the submission should be displayed (e.g., in the same window, a new tab, or a specific frame). It works together with the `action` attribute to determine the user experience after form submission.

By default, when a form is submitted, the browser collects all form field data and sends it to the URL specified in the action attribute using the indicated HTTP method, then loads the server’s response in a new or refreshed page. This standard behavior occurs unless JavaScript is used to intercept the submission.

Developers often override the default submit behavior using JavaScript—such as by calling `event.preventDefault()` in an `onsubmit` handler—to perform client-side validation before sending data. This allows for immediate feedback if required fields are missing or input is invalid, improving usability and reducing unnecessary server requests. Another common reason is